

Практическое занятие №1

Структура программы на языке JavaScript. Константы и переменные

Цель работы: глубоко изменить базовый синтаксис JS, объявление переменных (let, const), типы данных и их преобразование. Отработать навык работы с несколькими взаимосвязанными значениями

Время: 2 часа.

Лабораторное оборудование: персональный компьютер с доступом в Интернет.

Краткие теоретические сведения

Переменная – это именованный контейнер для данных (чисел, строк, объектов и т.д.), значение которого в ходе выполнения программы можно менять.

Объявление переменных:

– С помощью ключевого слова **var**: **var count = 0**;. Переменные, объявленные через var, имеют функциональную область видимости – они доступны во всей функции, в которой были созданы (**НЕ рекомендую использовать переменные через ключевое слово var**).

– С помощью ключевого слова **let**: **let name = "Ivan"**;. В отличие от **var**, **let** имеет блочную область видимости: переменная доступна только внутри блока кода (например, в цикле или условной конструкции), в котором она была объявлена.

Важные нюансы:

– **Имя переменной (идентификатор)** должно начинаться с буквы, символа подчёркивания **_** или знака доллара **\$**. Далее могут идти буквы, цифры, подчёркивания и доллары.

– JavaScript чувствителен к регистру: **count** и **Count** – это разные переменные.

– Нельзя использовать зарезервированные ключевые слова языка (например, **var**, **let**, **if** и т.д.) в качестве имён переменных.

– Если переменная объявлена, но не инициализирована (ей не присвоено значение), её значением по умолчанию будет **undefined**.

Константа – это переменная, значение которой после объявления нельзя изменить.

Для их объявления используется ключевое слово **const**.

Важные нюансы:

- Константу обязательно нужно **инициализировать** при объявлении – попытка объявить константу без значения приведёт к ошибке.
- Попытка переназначить константу (например, **const PI = 3.14; PI = 3;**) вызовет ошибку **TypeError**.
- **const** запрещает переназначение самой переменной (то есть присвоение новой ссылки), но если константа ссылается на объект или массив, то **внутренние свойства этого объекта или элементы массива можно менять**.

Соглашение об именовании: для глобальных констант (тех, что задают фундаментальные параметры всей программы) принято использовать стиль **UPPER_SNAKE_CASE** (все буквы заглавные, слова разделены подчёркиваниями). Это не требование языка, а просто сигнал для других разработчиков, что значение не должно меняться.

Что выбрать: let или const?

- Используйте **const** по умолчанию: если вы уверены, что значение не будет меняться в ходе выполнения программы, объявите его как константу. Это повышает читаемость и безопасность кода.
- Переходите на **let**, только если точно знаете, что значение будет переприсваиваться (изменяться по мере работы вашей программы).

Шаблонные строки заключаются в обратные кавычки (``...``) вместо одинарных или двойных. Внутри них для подстановки значений используется конструкция **`${выражение}`**. Результат вычисления этого выражения подставляется в строку.

Например:

```
const name = 'Анна';
const greeting = `Привет, ${name}!`; // В строку вставится значение переменной name
console.log(greeting); // Выведет: Привет, Анна!
```

С краткими теоретическими сведениями более подробно можно ознакомиться по ссылкам [Переменные](#), [Константы](#), [Шаблонные строки](#) – рекомендую использовать данный ресурс для изучения материала. Там вы найдёте всё что связано с **HTML**, **CSS** и **JavaScript** (и многое другое...) – [doka.guide](#)

Задания на выполнение практической работы

Этап 1: Базовый блок

Сценарий: вы разрабатываете модуль для интернет-магазина, отвечающий за отображение информации о товаре и расчёт скидки.

Задание 1.1 Объявление переменных и работа с const

1. Создайте пустую папку в любом удобном для вас месте (например, рабочий стол)
2. Зайдите в среду разработки **Visual Studio** и с помощью команды Файл → Открыть папку откройте созданную папку.
3. Создайте файл «**index.html**» с базовой структурой HTML (Введите «!» → Enter)
4. Создайте ещё один файл «**base_task_1.js**».
5. Подключите к файлу «**index.html**» ваш JS-скрипт из файла «**base_task_1.js**»
6. В файле со скриптом объявите константу **PRODUCT_NAME**, которая хранит название товара (например, «Смартфон»).
7. Объявите константу **PRODUCT_ARTICLE**, которая хранит артикул товара (например, «A1278G5»).
8. Попробуйте изменить значение одной из констант после ее объявления. Закомментируйте эту строку и напишите комментарий, объясняющий, какая ошибка возникла и почему.

Задание 1.2 Объявление переменных и работа с let

9. Объявите переменную **productPrice**, которая хранит исходную цену товара (например, 49999).
10. Объявите переменную **discountRate**, которая хранит размер скидки в процентах (например, 15).
11. Объявите переменную **isInStock**, которая хранит булево значение, есть ли товар на складе (true/false).
12. Объявите переменную **shippingCost**, которая хранит стоимость доставки (например, 500). Предположим, что доставка бесплатна при покупке от 10 000 руб.

Задание 1.3 Вычисления и преобразования типов

13. Рассчитайте цену со скидкой (**discountedPrice**) по формуле:

$$\text{productPrice} - (\text{productPrice} * \text{discountRate} / 100)$$

14. Используя **typeof**, выведите в консоль тип данных для каждой из созданных переменных (**PRODUCT_NAME**, **productPrice**, **isInStock**).

15. Преобразуйте значение **discountRate** в строку и сохраните в переменную **discountString**. Снова выведите ее тип в консоль, чтобы убедиться в преобразовании.

Задание 1.4 Логика и вывод с использованием шаблонных строк

16. Напишите условный оператор (**if**), который проверяет, является ли итоговая цена со скидкой больше или равной 5000.

17. Если условие истинно, перезапишите переменную **shippingCost** значением 0.

18. Используя шаблонные строки (бэктики ``...``), сформируйте и выведите в консоль информационное сообщение в формате:

```
Товар: [Название] (Арт.: [Артикул])
Цена: [Цена со скидкой] руб. (скидка [Размер скидки] %)
[Сообщение о наличии]: [Есть в наличии/Нет в наличии]
Стоимость доставки: [Стоимость доставки] руб.
```

Замените текст в квадратных скобках значениями переменных.

Для подстановки наличия используйте тернарный оператор «?»:

`isInStock ? «Есть в наличии» : «Нет в наличии».`

Пример итогового вывода в консоль:

```
string
number
boolean
string
Товар: Смартфон (Арт.: A1278G5)
Цена: 42499.15 руб. (скидка 15%)
Наличие: есть в наличии
Стоимость доставки: 0 руб.
```

Этап 2: Проектный блок (Индивидуальное задание)

Цель: описать сущности вашего проекта с помощью переменных.

Задание 2.1 Создание переменных и констант к индивидуальному проекту

1. Создайте файл **«project_data.js»**.
2. Подумайте, какие ключевые объекты и данные будут в вашем проекте.
3. Объявите не менее 5-7 переменных и констант, которые будут моделировать эти данные для вашего проекта. Давайте им осмысленные имена на английском языке. Среди них должны быть как примитивные типы (числа, строки, булевы значения), так и константы.

Пример для «Системы бронирования переговорных комнат»:

```
const MAX_BOOKING_HOURS = 4; // Максимальное время бронирования
const ROOM_NAME = "Alpha";
let bookingStartTime = 14; // Час начала брони
let bookingDuration = 2; // Длительность в часах
let isRoomAvailable = true;
let requiredSeats = 5;
let hasProjector = true;
```

Контрольные вопросы

1. Объясните, почему попытка изменить значение, объявленное с помощью `const`, приводит к ошибке? Какой именно это тип ошибки и в какой момент она возникает?
2. В задании вы использовали оператор `typeof`. Каков будет результат выполнения `typeof null` и почему этот результат считается исторической ошибкой в языке JavaScript?
3. В чем практическое преимущество использования шаблонных строк (бэктиков ``...``) по сравнению с обычной конкатенацией строк с помощью оператора `+`? Приведите пример из вашего кода.
4. Предположим, переменная `discountRate` получила свое значение из поля ввода на странице и имеет строковый тип. Почему расчет `discountedPrice` по вашей формуле мог бы дать неверный результат? Как бы вы исправили эту ситуацию?
5. Когда уместно использовать `const`, а когда `let`? Руководствуйтесь не только правилом «меняется/не меняется», но и концепцией ментальной нагрузки программиста.

Содержание отчёта

Отчет выполняется единым, в текстовом (электронном) виде, который содержит номер лабораторной работы, тему и цель работы, а также краткое описание выполняемых операций и скриншоты результатов выполнения, ответы на контрольные вопросы, выводы. Отчёт оформляется согласно ГОСТ 7.32-2017.

Структура отчёта должна содержать следующее:

- наименование и номер работы;
- тему и цель работы;
- раздел «Постановка задачи», где описываются задачи, которые необходимо решить в ходе выполнения практической работы;
- раздел «Ход работы», где описывается ход выполнения работы с кратким описанием выполняемых операций и приведенных результатов выполнения в виде скриншотов и листинга кода;
- ответы на контрольные вопросы (при наличии);
- выводы.